HITAN UWB · DW1000 SS-TWR FIRMWARE

Consolidated Firmware Code Changes

110 kbps long-range migration, errata workarounds, the Rev-D receive-path fix, and multi-station UWB backoff — line-exact, grounded in the Decawave reference driver, DW1000 User Manual, and application notes.

Scope: Tag + Anchor `dw1000.cpp` / `.h` ; §8 also `main.cpp` , `uart_communication.c`

Reference tree: `documentation/fw_actual_ref/` (§8: 2026-09-23 anchor backup, `sC3.0.67`)

Scheme: SS-TWR · Ch 2 · PRF 64M · PLEN 2048 · PAC64 · code 9 **Date:** 2026-06-09 · **§8 added:** 2026-10-13

> **Read first — the Rev-D long-range problem is a RECEIVE problem, not a logic problem.** Bench A/B test: **Rev-D TX → Skylab RX works at 2 km+**, but **Skylab TX → Rev-D RX fails at ~400 m**. A directional fault rules out the TWR math, the protocol handshake, and timing (those would be symmetric). The fault is in Rev-D's receive chain. The leading firmware cause is **§3 (GPIO clocks never enabled → external RX LNA never switched in)** — verify it on the bench *before* assuming a hardware redesign.

Severity: BLOCKER ranging wrong/broken · IMPORTANT quality / range margin · OPTIONAL choice / verify. Each change cites the exact current line; "→" shows the replacement.

§1 110 kbps data-rate migration BLOCKER
§2 TX-1 errata — delayed-TX clock IMPORTANT
§3 Rev-D RX deafness — GPIO/LNA BLOCKER
§4 Long-range link-budget hardening IMPORTANT
§5 Robustness / verify OPTIONAL
§7 LDE algorithm tuning (weak-signal) EXPERIMENT
§8 Multi-station UWB backoff (two systems, one tag) BLOCKER
§6 Bench checklist + at-a-glance

§1 110 kbps data-rate migration

Switches the radio from 850 kbps to 110 kbps for greater RX sensitivity / range. Apply **identically on Tag and Anchor** — UM Table 21: the receiver must use the same data-rate/SFD config as the transmitter, or ranging stops entirely.

> **Preamble length: go to 2048, not 1024.** UM **Table 57 (Recommended preamble lengths)**: 110 kbps → **2048 or 4096** (850 kbps → 256/512/1024; 1024 is the *top of the 850k range*, below the 110k floor). For a max-range 110k system use **PLEN_2048**, which per UM **Table 6** requires **PAC64**. The firmware author already anticipated this — the commented block at `anc/inc/dw1000.h:133–144` is exactly DWT_PLEN_2048 / DWT_PAC64 / DWT_BR_110K. A longer preamble adds airtime (drives the timeout values up, below) and slightly lengthens the round-trip — still far under the ~67 ms 32-bit wraparound limit (1B), so it is safe; just keep the reply delays tight.

1A BLOCKER Carrier-integrator multiplier — `anc/src/dw1000.cpp:296–298 — the root-cause fix for negative clockOffsetRatio`

```
/* current: 1024-sample constant used unconditionally */
clockOffsetRatio = dwt_readcarrierintegrator()
        * (FREQ_OFFSET_MULTIPLIER * HERTZ_TO_PPM_MULTIPLIER_CHAN_2 / 1.0e6);

/* → data-rate-aware: 110k accumulates over 8192 samples, not 1024 (8x larger) */
double freqOffsetMultiplier =
        (config.dataRate == DWT_BR_110K) ? FREQ_OFFSET_MULTIPLIER_110KB
                                         : FREQ_OFFSET_MULTIPLIER;
clockOffsetRatio = dwt_readcarrierintegrator()
        * (freqOffsetMultiplier * HERTZ_TO_PPM_MULTIPLIER_CHAN_2 / 1.0e6);
```

Basis: the two header macros differ only by divisor (1024 vs 8192); UM §7.2.40.11 states $N_{samples}$ = 8192 for 110 kb/s, 1024 otherwise . `config` is the file-static `dwt_config_t` from the header (in scope). 850k/6.8M correctly fall through to the 1024 macro — which is why 850k works today.

1B BLOCKER Reject non-physical ToF — `anc/src/dw1000.cpp:309–312 (you already removed abs())`

```
/* current (after your edit): negative tof now propagates as negative distance */
dwmInfo.tof = ((rtd_init - rtd_resp) / 2.0) * DWT_TIME_UNITS;
dwmInfo.distance = dwmInfo.tof * SPEED_OF_LIGHT * 100;
dwmInfo.dw_ready = VALID_DATA;

/* → guard: a negative result means a clock/wraparound glitch — drop it, don't report it */
double tof = ((rtd_init - rtd_resp) / 2.0) * DWT_TIME_UNITS;
if (tof < 0) {
    dwmInfo.dw_ready = DW_NO_VALID_DATA;        // non-physical: discard
} else {
    dwmInfo.tof = tof;
    dwmInfo.distance = dwmInfo.tof * SPEED_OF_LIGHT * 100;
```

```
    dwmInfo.dw_ready = VALID_DATA;
    dwt_readdiagnostics(&dwmInfo.diagnostics);  dwmInfo.wrong_ID = false;
}
```

At 110k the round-trip sits closer to the 32-bit timestamp wraparound ceiling (~67 ms). Once 1A is correct, negatives are rare glitches — reject them rather than fold to a fake short range (or, minimal: just restore `abs()`).

1C–1H BLOCKER Config + timeout constants — `both files, apply symmetrically`

#	LOCATION	CURRENT	CHANGE TO
1C	anc/inc/dw1000.h:126 tag/src/dw1000.cpp:47	`DWT_BR_850K,`	`DWT_BR_110K,`
1D	anc/inc/dw1000.h:121 tag/src/dw1000.cpp:42	`DWT_PLEN_1024,`	`DWT_PLEN_2048,` — UM Table 57 recommends 2048/4096 for 110k.
1E	anc/inc/dw1000.h:122 tag/src/dw1000.cpp:43	`DWT_PAC32,`	`DWT_PAC64,` — UM Table 6: preamble >1024 → PAC64.
1F	anc/inc/dw1000.h:128 tag/src/dw1000.cpp:49	`1100` (sfdTO)	`2049` — `(2048+1+64−64)`, 110k 64-symbol SFD at PLEN_2048. Any value ≥2049.
1G	anc/inc/dw1000.h:231	`RESP_RX_TIMEOUT_UUS 20000`	`25000` — its comment already says "preamble 2048"; bump for multipath margin.
1H	tag/Inc/dw1000.h:84	`TIMEOUT_PACS 1024/32*2`	`(2048/64*4)` — recomputed for PLEN_2048/PAC64; ×4 widens the preamble-detect hunt for weak long-range signal.

Auto-handled by `dwt_configure()` — no code: `SYS_CFG_RXM110K`, `LDE_REPC÷8`, `DRX_TUNE0b`, `DRX_TUNE1b_110K`, and `DRX_TUNE2` (indexed by PRF×PAC — picks up PAC64 automatically). **Optional:** for the more-robust Decawave 64-symbol SFD set `nsSFD` from `0` → `1` at anc/inc/dw1000.h:125 and tag/src/dw1000.cpp:46 (the author's commented 2048 block uses nsSFD=1) — must match on both ends (UM Table 21).

§2 TX-1 errata — force TX clock before delayed transmit

2A IMPORTANT Enable TX clock before each delayed `dwt_starttx` — `tag/src/dw1000.cpp:241 & anc/src/dw1000.cpp:411`

```
/* current: delayed TX issued without the errata workaround */
ret = dwt_starttx(DWT_START_TX_DELAYED);

/* → force TX clock on first (PMSC_CTRL0 bits 5,4 = 1,0) */
dwt_write8bitoffsetreg(PMSC_ID, PMSC_CTRL0_OFFSET,
    dwt_read8bitoffsetreg(PMSC_ID, PMSC_CTRL0_OFFSET) | 0x20);  // TXCLKS = 125MHz
ret = dwt_starttx(DWT_START_TX_DELAYED);
```

Errata §3.1.3 (verbatim): *"If the TX clock is enabled (PMSC_CTRL0 bits 5,4 set to 1,0 - force TX clock on) before the delayed TX command is issued then the problem does not happen."* The stock `dwt_starttx()` does not do this — it is the wrapper's job and is currently absent. A silently-failed delayed TX means the responder never replies → lost exchange; costs frames at the range edge where you can least afford them. Affects both SS-TWR response transmissions.

§3 Rev-D receive deafness — enable GPIO clocks for the external LNA

> **Leading suspected root cause of the Rev-D long-range failure.** `dwt_setlnapamode(...|DWT_LNA_ENABLE)` configures **GPIO6 = EXTRXE** to switch in the external RX LNA — but the firmware **never calls `dwt_enablegpioclocks()`**, and `dwt_setleds()` enables only the de-bounce clock (GPDCE), not the GPIO clock (GPCE/GPRN). UM §7.2.39: *"the GPIO clocks need to be turned on, before enabling or disabling the GPIO mode or value … by setting GPCE and GPRN in PMSC_CTRL0."* If the GPIO write doesn't take effect, EXTRXE never asserts, the Rev-D LNA is bypassed, and the receiver is deaf — exactly matching **RX-only, board-specific** (Skylab's integrated front-end doesn't depend on EXTRXE).

3A BLOCKER Add `dwt_enablegpioclocks()` after `dwt_setlnapamode()` — `tag/src/dw1000.cpp:113 & anc/src/dw1000.cpp:93`

```
dwt_setlnapamode(DWT_PA_ENABLE | DWT_LNA_ENABLE);
dwt_enablegpioclocks();  // REQUIRED: sets PMSC_CTRL0 GPCE|GPRN so EXTRXE (GPIO6) actually drives the LNA
```

One line, both files. APS004 §2: GPIO6/EXTRXE *"goes high when the DW1000 is in receive mode"* and (reference schematic) drives the RX-path RF switch into the LNA. This fix is curative **only if Rev-D's LNA is gated by EXTRXE** — confirm with §6 before concluding it is firmware-fixable vs a passive RX-front-end (matching/NF) hardware deficit.

Honest caveat: that an un-clocked `GPIO_MODE` write *silently* fails is documented in the UM but not independently re-confirmed by us; treat §3 as a high-probability hypothesis to verify on the bench (§6), not a settled fact. TX works at 2 km because the TX external-PA control (EXTPA/EXTTXE) is driven by the digital TX sequencer path, which is why the failure is RX-only.

§4 Long-range link-budget hardening

4A IMPORTANT **TX power temperature compensation** — `dwConfig, anc/src/dw1000.cpp:80–106 & tag/src/dw1000.cpp:102–136`

TX power is set once at boot via `dwt_configuretxrf()`; `dwt_readtempvbat()` / `dwt_calcpowertempadj()` are never called. APS023 Pt 1 §8.2: channel-2 TX power falls **~0.035 dB/°C** → a −10 °C→+30 °C outdoor swing ≈ **1.4 dB** of lost EIRP, straight off the long-range budget. **Fix:** periodically (post-wake / on temperature delta) read chip temperature and re-apply temperature-compensated power before TX.

4B IMPORTANT **Per-device antenna-delay calibration** — `anc/inc/dw1000.h:149–150 ; tag/Inc/dw1000.h:92–93`

Antenna delays are hardcoded (anc 16550/16500; tag 16500/16500). APS014: uncalibrated antenna delay ≈ **30 cm 3σ** error, drifting **2.15 mm/°C**. This is a *constant* offset (doesn't grow with range, won't stop you reaching distance) but it is your absolute-accuracy floor — calibrate per unit if sub-30 cm accuracy matters.

§5 Robustness / verify (lower priority)

#	LOCATION	CHANGE	SEVERITY
5A	`anc/src/dw1000.cpp:320–330` `tag/src/dw1000.cpp ~255`	Error path clears only `ALL_RX_TO|ALL_RX_ERR` before `dwt_rxreset()`, leaving "good-RX" latched bits set. Clear all relevant status bits. Runs constantly at long range (frequent RX errors).	MEDIUM
5B	`anc/src/dw1000.cpp:274 vs :160`	`dwRange` uses `frame_len == sizeof()` while `dwScanMode` uses `<=`. Make consistent (prefer `<=`). Robustness only — **not** the "drops all frames" critical bug an earlier pass mis-rated (ranging works at 3 km today, so `==` is functioning).	LOW
5C	`anc/src/dw1000.cpp:190`	Backoff uses `random()`, likely unseeded → every anchor shares one sequence → synchronized collisions in multi-anchor setups. Seed per-device (e.g. from device ID). **Promoted to 8E** now that two stations share a tag.	→ 8E

5E	`anc/src/usart.c:379 (RxEventCallback; no ErrorCallback exists)`
	`tag likewise`
		UART receive dies for good on any line error. Reception runs as `HAL_UARTEx_ReceiveToIdle_DMA` and is re-armed only from `UART_GetMessage` after a good idle event. In DMA mode the HAL treats *every* line error (framing, noise, overrun, parity) as blocking: it aborts the receive and calls `HAL_UART_ErrorCallback` — which this firmware does not define, so nothing re-arms. One glitch from a flaky USB adapter, a hot-plug, or a host opening the port at the wrong baud, and the anchor answers nothing until power-cycled (the main loop and watchdog keep running; only the UART is deaf). Bench 2026-10-13: right after one 3-byte Status frame was sent at the wrong baud (115200 instead of 921600), all three Mob anchors stopped answering at the correct baud, with no USB re-enumeration in the kernel log, and a power-cycle restored them. The field symptom is an anchor that "does not answer on USB" at BIT with its adapter enumerated fine.
		`void HAL_UART_ErrorCallback(UART_HandleTypeDef *huart) { if (huart->Instance == USART6 || huart->Instance == USART3) { __HAL_UART_CLEAR_FLAG(huart, UART_CLEAR_OREF | UART_CLEAR_NEF | UART_CLEAR_FEF | UART_CLEAR_PEF); uartIdle = false; UART_RX_len = 0; HAL_UARTEx_ReceiveToIdle_DMA(huart, UART_Buff, sizeof(UART_Buff)); // re-arm: a line error ends one frame, not reception } }`
		The partial frame is dropped (the host times out and resends). Same for the tag's UART.
			BLOCKER

5D	`anc/src/dw1000.cpp:110–115`	LDE noise-threshold / peak-multiplier tuning for weak-signal first-path recovery — see **§7** (the `NEW_NTM` block is the hook, but its current value is a no-op; §7 gives the real values).	→ §7

§7 LDE algorithm tuning — weak-signal first-path detection

> **Do §3 (RX path) first.** LDE operates on signal that already reached the receiver. If Rev-D's LNA isn't switched in, no LDE value recovers a signal that is below the noise floor. Tune LDE *after* the receiver is actually hearing — to squeeze the last few dB of weak-signal margin, not as the cure for the 400 m failure. Hence EXPERIMENT, not blocker.

What the firmware runs today (verified): `dwt_configure()` calls the stock `_dwt_configlde()`, which writes `LDE_CFG1 = LDE_PARAM1 = PEAK_MULTPLIER(0x60) | N_STD_FACTOR(13) = 0x6D` (PMULT=3, NTM=13) and `LDE_CFG2 = 0x0607` for 64 MHz PRF. The `#ifdef NEW_NTM` block at anc/src/dw1000.cpp:110–114 is compiled

out — **and even if enabled it re-writes the same `0x6D`**, so it is a no-op-in-waiting, *not* the NLOS tuning its comment implies.

7A EXPERIMENT Lower the LDE first-path threshold (NTM + PMULT) — `write after dwt_configure(), e.g. anc/src/dw1000.cpp:110 / tag/src/dw1000.cpp ~118`

`LDE_CFG1` (reg 0x2E:0806) is one byte = **PMULT** (bits 7:5) + **NTM** (bits 4:0). Your current `0x6D` is tuned for *close-up line-of-sight accuracy* — the opposite of what a weak long-range first path needs.

LDE_CFG1	PMULT	NTM	MEANING (UM §7.2.47.2, VERBATIM GUIDANCE)
`0x6D` **(now)**	3	13	*"used … to give more accuracy in close-up LOS conditions."* Highest threshold — most likely to miss a weak/attenuated first path.
`0x6C`	3	12	Decawave default — *"reasonable compromise … between falsely triggering on noise peaks and missing real attenuated first paths."*
`0x07` **(NLOS)**	0	7	NTM: *"Where NLOS performance is more important … the default NTM value of 0x07 … would be a better choice."* PMULT: *"For NLOS operations these bits should be set to 0."*

```
/* after dwt_configure() — it calls _dwt_configlde() and would overwrite an earlier write */
/* sweep this value on the bench; lower = detects weaker first paths, but risks noise lock */
dwt_write8bitoffsetreg(LDE_IF_ID, LDE_CFG1_OFFSET, 0x07);  // PMULT=0, NTM=7 (NLOS / long-range)
```

Sweep, don't slam: `0x6D` (now) → `0x6C` (default) → `0x07` (NLOS). Measure first-path detection rate and range scatter at 400 m / long range at each. Too low a threshold makes LDE **lock onto noise** → false detects and garbage ranges → *more* effective loss. APS006 Part 2/3 is the tuning reference.

Placement: must be written *after* `dwt_configure()` (which sets the LOS default 0x6D), on the anchor that is the existing `NEW_NTM` location. Apply identically on Tag and Anchor. **Live-tunable from rf_analyzer ✓** — `LDE_CFG1` is at reg 0x2E, sub-offset 0x0806. The firmware WRITE opcode (14) carries a **16-bit** reg_offset (same struct as the READ opcode, whose ACC_MEM reads already use offsets >0xFF in production), so 0x0806 is reachable. `commands.UWB_write8bit_raw` now sends the offset as little-endian [lo,hi]; the analyzer Configuration tab has a "Live LDE tuning" control to sweep 0x6D/0x6C/0x07 in the field without re-flashing. (Earlier note said this needed a flash — corrected: only LDE_REPC math is driver-side; the byte itself is pokeable live, but resets to 0x6D on any dwConfig() re-run.)

Related, NOT a tuning knob: `LDE_REPC` (reg 0x2E:2804) must be divided by 8 at 110 kbps — `dwt_configure()` does this automatically (`reg16 >>= 3` for `DWT_BR_110K`). Leave it alone; do not hand-write it.

§8 Multi-station coexistence — UWB backoff (two anchor systems, one tag)

Line numbers in this section reference the anchor source in `firmware_code_for_reference/anchor-tag-bootload-backup-23-9-2026.zip` (`Anchor/Core/…`, firmware `sC3.0.67`), which is the build that carries opcode 19 (`BACKOFF_CTRL`). Added 2026-10-13.

Goal. Two stations with different `PC_ID` (e.g. Mob + Fly) range the *same* tag. The tag answers whichever anchor polls it (`tag/…/dw1000.cpp:253` checks only `des == SELF_ID`, no pc_id filter), so the only shared resource is the air. The anchor already implements a passive **listen-before-talk**: between UART commands it sits in `dwScanMode()` (main.cpp:408, gated by `BackOFF_CTRL`); any frame whose pc_id (`src>>2` for anchor frames, `des>>2` for tag frames) is foreign arms `backoff_timer = now + random()%200 + 10` (dw1000.cpp:201, i.e. 10–209 ms); a `SEARCH_TWR` that arrives while it is armed skips `dwRange()` and answers with `errors.bit4 = UWB_Backoff` (main.cpp:252, uart_communication.c:176). The PC side will treat that bit as "not polled" and yield. The mechanism is sound **but as written it starves the second station** — §8A.

Starvation (why 8A is a blocker). dw1000.cpp:201 *re-arms* the timer on **every** foreign frame. Station A ranging at ~30 Hz puts a foreign frame on the air every ~11 ms at B's anchors; each one pushes B's deadline ≥10 ms further out, so B's backoff *never expires* (P(random ≤ 11 ms) ≈ 1%). B reports `UWB_Backoff` forever, never transmits, so A's anchors never hear B and A never yields. Whoever is already running wins permanently — in both directions, after any handover. A passive backoff can only hand the channel over if the waiting side is *guaranteed* to transmit after a bounded wait (classic CSMA). Fix: arm once per busy period, expire, transmit.

Bench result, firmware sC3.0.69 (2026-10-13, Fly PC id 1 + Mob PC id 11, tag 56).

- **8B works:** opcode 19 = 0 releases an armed backoff at once and the anchor ranges on the next request; re-enabling does not re-arm by itself.
- **Arming works:** five rounds of the other station's traffic arm all three anchors.
- **Expiry is broken — blocker:** the Status backoff field (now ms left) counts down and **stops at 1**; the anchor answers every SEARCH_TWR with bit 4 for good (watched 3 s, and indefinitely in earlier runs). One burst of foreign traffic silences a station permanently. Likely an off-by-one in the countdown or in the expiry compare (`<` vs `<=`, or a decrement that stops at 1).
- **8C half done:** Status carries ms left; the TWR reply's distance field is still 0 on a backed-off frame, so the PC cannot size its yield.

PC-side workaround in place: an anchor that reports backoff for longer than 250 ms (+0–150 ms jitter) is released with opcode 19 off/on and polled again. Measured, both stations on one tag for 20 s: **42 + 30 valid ranges/s** (each alone: 81–89/s), p95 gap 83 / 229 ms; with backoff off: 13 + 10/s, p95 gap 431 / 368 ms, max 3.6 s. Field scenario (one station alone 10 s, the other joins for 20 s, the first stops), both orders: the newcomer's first range after 0.4–1.4 s; while both run 21–48 valid ranges/s each, worst gap 0.7–1.9 s; the station left alone is back to full rate (85–90/s) within the first second. Releasing a whole station at once instead of per anchor measured worse (worst gaps 5–6 s). With expiry fixed, the release becomes the rare §8A bounded wait instead of the normal path, and the fine window of 8D can interleave the stations per exchange, which is what will shorten those overlap gaps.

Bench result, firmware sC3.0.70 (2026-10-14, same rig). Expiry fixed: after a burst of the other station's traffic the Status field read 42–44 ms left, counted down to 0, and the anchor ranged again by +134 ms. **TWR reply:** carries the backoff bit only, by design (no time field; only Status counts down). That is all the PC needs: it polls every round anyway, so the next reply shows when the backoff has ended. Measured (both stations ranged lower alone than the day before, 60–68/s, so compare within the day): simultaneous start 30.6 + 31.5 valid ranges/s, p95 gap 111 / 90 ms (backoff off: 14.1 + 8.7/s); join/leave both orders — newcomer's first range 0.4–0.75 s, 27–45/s each while both run, worst gap 0.8–1.9 s, full rate again within 1 s. A 30 ms PC yield was tried and was worse (fewer whole rounds). The remaining overlap gaps are slot-like alternation; the fine window of 8D is the next firmware lever. (PC side since: taking turns, ON/OFF 250/250 — Mob's share 0.3–1.1 → 7.3–8.0 valid ranges/s with Mob ranging from its front anchor only.)

8A BLOCKER Arm the backoff once per busy period; own-pc_id frames must not clear it — `anc/src/dw1000.cpp:198–202 (dwScanMode)`

```
/* current: every foreign frame re-arms (never expires under continuous foreign traffic);
   every OWN frame clears it (A's own next poll erases the yield B just earned) */
if (dwmInfo.detec_pc_id == true){
    dwmInfo.backoff_timer = 0;
}else{
    dwmInfo.backoff_timer = _1ms_tick + random() % TIME_TOP_RANGE_MS + MIN_BACKOFF_MS;
}

/* → arm ONLY when not already running; a running timer is left alone by both own and foreign frames.
   "Running" = deadline still in the future; an expired deadline counts as idle so it can re-arm on the next busy period. */
bool running = (dwmInfo.backoff_timer != 0) && (dwmInfo.backoff_timer > _1ms_tick);
if (!dwmInfo.detec_pc_id && !running) {
    dwmInfo.backoff_timer = _1ms_tick + random() % TIME_TOP_RANGE_MS + MIN_BACKOFF_MS;
}
```

Bounded wait: at most `MIN_BACKOFF_MS + TIME_TOP_RANGE_MS − 1` after the first foreign frame, the next `SEARCH_TWR` goes out regardless (main.cpp:252 already clears the timer on expiry and ranges). That poll is what the *other* station's anchors hear, back off from, and report to their PC, which then yields. Without the own-pc_id change, an anchor that hears a sibling anchor's poll (same pc_id, every round) drops a foreign backoff in progress, so a running station never actually yields even once its PC wanted to. Collision risk on the barge-in poll is ~30% (air occupancy of one station ≈ 3×(3–4 ms) per 33 ms round); a collided poll is a normal miss and the next one goes out after the next backoff.

8B BLOCKER `BACKOFF_CTRL = 0` must also release an armed timer — `anc/src/main.cpp:281–283 (UART_OPCODE_BACKOFF_CTRL)`

```
/* current: disables the scan only — an already-armed deadline keeps blocking SEARCH_TWR (main.cpp:252) until it expires */
if(msg.op_backoff_ctrl_req.BackOFF_CTRL == 0){
    BackOFF_CTRL = false;
    dwmInfo.backoff_timer = 0;   // → release: "disable backoff" means "poll on the next SEARCH_TWR"
}
```

This is the PC's **barge-in** lever: after N consecutive `UWB_Backoff` replies the PC sends opcode 19 = 0 to one anchor, polls once (making the other station yield), and sends 19 = 1 again. With 8A in place barge-in is rarely needed; without 8A it is the *only* way for the second station to get in, and today it costs an extra ≤209 ms wait because the armed deadline survives the disable. One line.

8C below is dropped (2026-10-14): the PC uses only the backoff bit of the TWR reply and needs no time — it polls every round, so the next reply shows when the backoff ended. sC3.0.70 already clears the bit on expiry and counts down in Status. Kept for the record.

8C DROPPED Report backoff *time left* (ms), not a raw tick, and only while it is running — `anc/src/uart_communication.c:176 (TWR reply) · :193 (Status) · :108 / :128 (BIT, wake-up)`

```
/* current (TWR reply): bit = "timer non-zero" — also set by a STALE, already-expired deadline that no SEARCH_TWR has cleared yet;
   uwb_distance on a skipped range carries the PREVIOUS exchange's distance; Status.UWB_backoff is the absolute _1ms_tick deadline (meaningless to the PC) */
reply->op_twr_rply.errors.bits.UWB_Backoff = !!dwmInfo.backoff_timer;

/* → one helper, used by all four replies */
static uint32_t backoff_left_ms(void) {
    return (dwmInfo.backoff_timer > _1ms_tick) ? (dwmInfo.backoff_timer - _1ms_tick) : 0;
}
/* TWR reply (:176): */
uint32_t left = backoff_left_ms();
reply->op_twr_rply.errors.bits.UWB_Backoff = (left != 0);
if (left) reply->op_twr_rply.uwb_distance = left;   // skipped range: distance field carries ms-left instead (PC reads it only when bit4 is set)
/* Status reply (:193): */
reply->op_stt_rply.UWB_backoff = backoff_left_ms();
/* BIT (:108) and wake-up (:128) replies: */
reply->…errors.bits.UWB_Backoff = (backoff_left_ms() != 0);
```

Wire-compatible: no struct change, no length change; `uwb_distance` is `uint32_t` and otherwise 0/stale on a skipped range (`data_status` already says no DW data). The PC then waits exactly `left` ms (+ a few ms) instead of a blind 300 ms, and never mistakes a stale deadline for a live backoff. Keep the existing convention that the reply is still sent (the anchor must answer every `SEARCH_TWR`, backoff or not).

8D CHOOSE Backoff window: coarse (TDM-like, 300 ms slots) or fine (exchange-level interleave) — `anc/inc/dw1000.h:93–94`

MODE	CONSTANTS	BEHAVIOUR WITH TWO STATIONS	PC SIDE
Coarse (today's values)	`TIME_TOP_RANGE_MS 200` `MIN_BACKOFF_MS 10`	Stations alternate in bursts: one ranges at ~30 Hz until the other barges in (≤209 ms after it first hears it), then yields. Each station sees ~9-round holes of ~300 ms per handover.	On bit4: wait the PC's yield (Advanced, default 300 ms; 40 ms while taking turns) and poll again; a long backoff is released via 8B.
Fine (recommended)	`TIME_TOP_RANGE_MS 15` `MIN_BACKOFF_MS 1`	Backoff on the scale of one exchange (~3–4 ms air, ~11 ms with UART). Stations interleave per exchange; each gets ~12–15 Hz with jitter and no holes. No handover protocol, no starvation, no yield timer. **Requires 8A** (with re-arm, a short window makes starvation worse, not better).	On bit4: re-send `SEARCH_TWR` to the same anchor after ~5 ms, up to 3 tries (a rejected request costs ~1 ms of UART), then move on.

The PC's position/bearing filters propagate with the measured `dt` (CV predict, dt-scaled gates), so either mode is safe for the filtering; the fine mode is better for it (jittered 15 Hz beats 30 Hz with 300 ms holes — fewer lost rounds, less lag at every handover). Both values are compile-time; a runtime setter (opcode 19 payload byte 2 = top-ms) would let the PC pick per deployment.

8E IMPORTANT Seed `random()` per anchor — `anc/src/main.cpp init (before the main loop); promotes 5C`

```
srandom((unsigned)(PC_ID ^ (dwmInfo.anchor_id << 8) ^ HAL_GetTick() ^ HAL_GetUIDw0()));
```

With two stations this is no longer cosmetic: all six anchors run the same unseeded sequence, so two anchors that hear the same foreign frame pick the same backoff and their barge-in polls collide deterministically, round after round. Any per-device seed breaks the lock-step.

8F IMPORTANT Imposter mode (BIT) must match the id a poll can actually carry — bit 15 masked — `anc/src/dw1000.cpp:402 (dwTagMode) · :40 (dwConfig, tx_resp_msg.src) · :232 (dwRange, rx_resp_msg.src) · main.cpp:248`

> **Latent: no station is known to hit this today.** It was suspected for Mob's failing BIT on 2026-10-13, but Mob's PC id read back as 11 (self ids 0x2C–0x2E), well clear of it. The station self-test switches each anchor to imposter mode and has the others range it under its Status `self_id = PC_ID<<2 | anchor_id`. `SEARCH_TWR` strips bit 15 of the requested id as the LoRa-only flag (main.cpp:248, `& 0x7FFF`), so the poll goes out with `des = self_id & 0x7FFF` — but the imposter accepts a poll only when `des == self_id` *unmasked*. Any station whose **PC id is ≥ 8192** (bit 15 of `self_id` set) would have anchors that can never be ranged as imposters. Keep every station's PC id below 8192.

```
/* current (dwTagMode): the imposter answers only its full 16-bit self id */
if (rx_buffer.des == tx_resp_msg.src) {

/* → the id a poll can carry: bit 15 is the LoRa-only flag on the UART side, never part of an over-the-air id */
if (rx_buffer.des == (tx_resp_msg.src & 0x7FFF)) {

/* and in dwRange, the expected responder id is already the masked tag_dest (rx_resp_msg.src = dwmInfo.tag_dest) —
   the imposter's reply must carry the same, so build its response src masked too: */
tx_resp_msg.src = ((dwmInfo.mine_pc_id << 2) | dwmInfo.anchor_id) & 0x7FFF;   // dwConfig, :40
```

Two masks, both sides of the same exchange; a real tag is unaffected (its id never has bit 15). **Until this lands**, the field workaround is a PC id below 8192 on the affected station (`SET_ID`, hitan-cli menu 2) — the PC-side BIT now names this condition from the Status `self_id` instead of reporting three unrangeable pairs. Longer term, bit 15 of the UART id should not double as a flag at all: 14 bits of PC id + 2 of anchor id already fill the 16, so the LoRa-only request belongs in its own byte.

8G IMPORTANT SET_ID must also refresh the UWB frame ids, or say "reboot required" — `anc/src/main.cpp (case UART_OPCODE_SET_ID) · dw1000.cpp:36–40 (dwConfig)`

The changelog (C1.6.03) says a PC-id change no longer needs a reset. That holds only for the Status `self_id`: the handler calls `SetAnchor_self_id()`, which rebuilds `lora_rxInfo.self_id`. The UWB side keeps the boot-time value — `dwmInfo.mine_pc_id` is set once by the static initializer, and `tx_poll_msg.src`, `rx_resp_msg.des` and `tx_resp_msg.src` are built from it once in `dwConfig()`. Until a power-cycle the anchor **reports the new id but ranges, answers BIT as imposter, and judges backoff under the old one**: a BIT run straight after SET_ID polls the imposter at an id it does not answer, and every pair fails.

```
/* SET_ID handler, after a successful Flash_Write: */
SetAnchor_self_id();
dwmInfo.mine_pc_id = PC_ID;
uint16_t id = (dwmInfo.mine_pc_id << 2) | dwmInfo.anchor_id;
tx_poll_msg.src = rx_resp_msg.des = tx_resp_msg.src = id;   // the three frame templates dwConfig() fills
```

Also on the PC side: the payload is the PC id itself (the anchor id comes from the cable pins). The hitan-cli SET_ID menu used the older `pc<<2 | anchor` packing, so entering "PC 2, anchor 3" stored PC id 11; fixed 2026-10-13, and it now prompts for a power-cycle.

PC-side contract (hitan_station), for reference. `errors` bit4 (0x10) on a TWR reply means *not polled*: not a missed range, not a dead anchor, not a silent tag (no LoRa wake-up, no spare engagement, no miss count). On bit4 the PC waits its yield (Advanced, default 300 ms; 40 ms while taking turns) and polls again — the next reply shows whether the backoff has ended; an anchor backed off past the release time is freed via opcode 19 off/on (8B). Stations must have distinct `PC_ID` (bits 15:2 of `self_id`, set with `SET_ID`) — verify on both stations before the first joint test.

§6 Bench verification & at-a-glance

Confirm §3 (RX deafness) in 5 minutes, before changing anything else. After `dwConfig()` runs, read `GPIO_MODE` (0x26:00) bits 19:18 (MSGP6): if they read **00** despite `dwt_setlnapamode` writing **01**, the GPIO write silently failed — that is the smoking gun. Also read `PMSC_CTRL0` bits 16/17 (expect **0/0** now, **1/1** after 3A). Scope GPIO6/pin 30 during RX (flat-low now → pulses high after 3A). Then apply 3A and re-run Skylab TX → Rev-D RX at 400 m. If loss drops toward 0–4%, confirmed firmware-fixable; if not, the deficit is in Rev-D's passive RX front-end (hardware).

DO FIRST (BLOCKERS)

- **3A** — GPIO clocks → RX LNA *(the Rev-D fix)*
- **1A** — 110k carrier-int multiplier
- **1B** — reject negative ToF
- **1C–1F** — BR_110K + PLEN_2048/PAC64 + sfdTO, both ends
- **8A / 8B** — backoff: arm once; CTRL=0 releases *(two stations)*
- **5E** — UART error callback re-arms RX (deaf anchor)

THEN (RANGE MARGIN / MULTI-STATION)

- **2A** — TX-1 errata clock enable
- **4A** — TX power temp compensation
- **1G / 1H** — RX / preamble-detect timeouts
- **4B** — per-device antenna cal
- **8E** — seed backoff RNG
- **8F** — imposter id mask (latent; PC id ≥ 8192)

VERIFY / WHEN SCALING

- **5A** — clean status-bit clearing
- **5B** — `==` / `<=` consistency
- **8D** — backoff window: coarse vs fine *(needs 8A)*
- **7A** — LDE NTM/PMULT sweep *(after §3)*

Verdict on the TWR logic: sound. The directional A/B result (Rev-D→Skylab works, Skylab→Rev-D fails) is itself proof that the SS-TWR exchange, addressing, and timing are correct — a logic defect would be symmetric. The accuracy items (1A, 4B) affect distance *values*, not whether frames are received; the genuine logic nits (5B, 5C) don't cause the long-range loss. The long-range loss is a Rev-D *receive* problem — §3.

Line numbers reference `documentation/fw_actual_ref/`. Grounded in: DW1000 User Manual §7.2.39 / §7.2.40.11 / §7.2.47.2 / Tables 6, 21, 57 · DW1000 Errata §3.1.3 · APS004 §2–3 · APS006 Pt 2/3 · APS011 · APS014 · APS017 · APS023 Pt 1 §8.2. The reference tree is read-only; apply these in your working firmware source.